

Q1. Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

```
mysql> -- a) Selection: employees in IT department
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT * FROM Employee WHERE DeptID = 1;
+-----+-----+-----+-----+
| EmpID | EName | Salary | DeptID |
+-----+-----+-----+-----+
| 101   | Arun  | 55000.00 | 1 |
| 102   | Bala  | 48000.00 | 1 |
| 109   | Indra  | 58000.00 | 1 |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)

mysql> -- b) Projection: only names and salaries
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT EName, Salary FROM Employee;
+-----+-----+
| EName | Salary |
+-----+-----+
| Arun  | 55000.00 |
| Bala  | 48000.00 |
| Chitra | 62000.00 |
| Divya | 45000.00 |
| Esakki | 70000.00 |
| Farook | 39000.00 |
| Gowri | 52000.00 |
| Harish | 41000.00 |
| Indra  | 58000.00 |
| Jaya  | NULL |
+-----+-----+
10 rows in set (0.000 sec)

mysql> -- c) Union: all DeptIDs appearing in either table
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT DeptID FROM Employee
-> UNION
-> SELECT DeptID FROM Department;
+-----+
| DeptID |
+-----+
| 1 |
| 2 |
| 3 |
| 4 |
+-----+
4 rows in set (0.000 sec)
```

```
mysql> -- d) Set Difference: departments with no employees
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT DeptID FROM Department
-> WHERE DeptID NOT IN (SELECT DeptID FROM Employee);
Empty set (0.000 sec)
```

```
mysql> -- e) Cartesian Product
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT * FROM Employee, Department;
```

EmpID	EName	Salary	DeptID	DeptID	DeptName
101	Arun	55000.00	1	4	Sales
101	Arun	55000.00	1	3	Finance
101	Arun	55000.00	1	2	HR
101	Arun	55000.00	1	1	IT
102	Bala	48000.00	1	4	Sales
102	Bala	48000.00	1	3	Finance
102	Bala	48000.00	1	2	HR
102	Bala	48000.00	1	1	IT
103	Chitra	62000.00	2	4	Sales
103	Chitra	62000.00	2	3	Finance
103	Chitra	62000.00	2	2	HR
103	Chitra	62000.00	2	1	IT
104	Divya	45000.00	2	4	Sales
104	Divya	45000.00	2	3	Finance
104	Divya	45000.00	2	2	HR
104	Divya	45000.00	2	1	IT
105	Esakki	70000.00	3	4	Sales
105	Esakki	70000.00	3	3	Finance
105	Esakki	70000.00	3	2	HR
105	Esakki	70000.00	3	1	IT
106	Farook	39000.00	3	4	Sales
106	Farook	39000.00	3	3	Finance
106	Farook	39000.00	3	2	HR
106	Farook	39000.00	3	1	IT
107	Gowri	52000.00	4	4	Sales
107	Gowri	52000.00	4	3	Finance
107	Gowri	52000.00	4	2	HR
107	Gowri	52000.00	4	1	IT
108	Harish	41000.00	4	4	Sales
108	Harish	41000.00	4	3	Finance
108	Harish	41000.00	4	2	HR
108	Harish	41000.00	4	1	IT
109	Indra	58000.00	1	4	Sales
109	Indra	58000.00	1	3	Finance
109	Indra	58000.00	1	2	HR
109	Indra	58000.00	1	1	IT
110	Jaya	NULL	2	4	Sales
110	Jaya	NULL	2	3	Finance
110	Jaya	NULL	2	2	HR
110	Jaya	NULL	2	1	IT

40 rows in set (0.000 sec)

```
mysql> -- f) Natural Join (via common DeptID)
Query OK, 0 rows affected (0.015 sec)

mysql> SELECT * FROM Employee E JOIN Department D ON E.DeptID = D.DeptID;
+-----+-----+-----+-----+-----+-----+
| EmpID | EName | Salary | DeptID | DeptID | DeptName |
+-----+-----+-----+-----+-----+-----+
| 101 | Arun | 55000.00 | 1 | 1 | IT |
| 102 | Bala | 48000.00 | 1 | 1 | IT |
| 109 | Indra | 58000.00 | 1 | 1 | IT |
| 103 | Chitra | 62000.00 | 2 | 2 | HR |
| 104 | Divya | 45000.00 | 2 | 2 | HR |
| 110 | Jaya | NULL | 2 | 2 | HR |
| 105 | Esakki | 70000.00 | 3 | 3 | Finance |
| 106 | Farook | 39000.00 | 3 | 3 | Finance |
| 107 | Gowri | 52000.00 | 4 | 4 | Sales |
| 108 | Harish | 41000.00 | 4 | 4 | Sales |
+-----+-----+-----+-----+-----+-----+
10 rows in set (0.004 sec)
```

Q2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

```
mysql> CREATE TABLE Department_Q2 (
->     DeptID    INT PRIMARY KEY,
->     DeptName  VARCHAR(50) NOT NULL UNIQUE
-> );
Query OK, 0 rows affected (0.056 sec)

mysql> CREATE TABLE Student_Q2 (
->     SID       INT PRIMARY KEY,
->     Name      VARCHAR(50) NOT NULL,
->     DOB       DATE NOT NULL,
->     DeptID    INT NOT NULL,
->     FOREIGN KEY (DeptID) REFERENCES Department_Q2(DeptID)
->     ON DELETE CASCADE
-> );
Query OK, 0 rows affected (0.024 sec)

mysql> DESC Department_Q2;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| DeptID | int | NO | PRI | NULL | |
| DeptName | varchar(50) | NO | UNI | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.042 sec)

mysql> DESC Student_Q2;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| SID | int | NO | PRI | NULL | |
| Name | varchar(50) | NO | | NULL | |
| DOB | date | NO | | NULL | |
| DeptID | int | NO | MUL | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.008 sec)
```


Q3. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

```
mysql> SELECT Category,
->      COUNT(*)      AS NumItems,
->      SUM(Amount)   AS TotalSales,
->      AVG(Amount)   AS AvgSale,
->      MAX(Amount)   AS MaxSale,
->      MIN(Amount)   AS MinSale
-> FROM Sales
-> GROUP BY Category
-> HAVING SUM(Amount) > 2000;
```

Category	NumItems	TotalSales	AvgSale	MaxSale	MinSale
Electronics	4	65400.00	16350.000000	55000.00	500.00
Apparel	3	4100.00	1366.666667	2200.00	700.00
Home Appliances	3	37600.00	12533.333333	32000.00	2100.00

3 rows in set (0.011 sec)

Q4. Write a correlated sub-query to find employees earning more than the average salary of their own department.

```
mysql> SELECT E1.EmpID, E1.EName, E1.Salary, E1.DeptID
-> FROM Employee E1
-> WHERE E1.Salary > (
->      SELECT AVG(E2.Salary)
->      FROM Employee E2
->      WHERE E2.DeptID = E1.DeptID
-> );
```

EmpID	EName	Salary	DeptID
101	Arun	55000.00	1
103	Chitra	62000.00	2
105	Esakki	70000.00	3
107	Gowri	52000.00	4
109	Indra	58000.00	1

5 rows in set (0.002 sec)

Q5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

```
mysql> -- INNER JOIN
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT E.ENAME, P.ProjName
-> FROM Employee E
-> INNER JOIN Works_On W ON E.EmpID = W.EmpID
-> INNER JOIN Project P ON W.ProjID = P.ProjID;
+-----+-----+
| EName | ProjName |
+-----+-----+
| Arun   | ERP Upgrade |
| Bala   | ERP Upgrade |
| Indra   | ERP Upgrade |
| Chitra | Recruitment Drive |
| Esakki | Audit 2025 |
| Gowri  | Q3 Sales Push |
+-----+-----+
6 rows in set (0.011 sec)

mysql> -- LEFT OUTER JOIN (all employees, even those on no project)
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT E.ENAME, P.ProjName
-> FROM Employee E
-> LEFT JOIN Works_On W ON E.EmpID = W.EmpID
-> LEFT JOIN Project P ON W.ProjID = P.ProjID;
+-----+-----+
| EName | ProjName |
+-----+-----+
| Arun   | ERP Upgrade |
| Bala   | ERP Upgrade |
| Chitra | Recruitment Drive |
| Divya  | NULL |
| Esakki | Audit 2025 |
| Farook | NULL |
| Gowri  | Q3 Sales Push |
| Harish | NULL |
| Indra   | ERP Upgrade |
| Jaya   | NULL |
+-----+-----+
10 rows in set (0.001 sec)
```

```
mysql> -- FULL OUTER JOIN
Query OK, 0 rows affected (0.000 sec)

mysql> -- MySQL has no native FULL OUTER JOIN; emulate with UNION of LEFT + RIGHT
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT E.ENAME, P.ProjName
-> FROM Employee E
-> LEFT JOIN Works_On W ON E.EmpID = W.EmpID
-> LEFT JOIN Project P ON W.ProjID = P.ProjID
-> UNION
-> SELECT E.ENAME, P.ProjName
-> FROM Employee E
-> RIGHT JOIN Works_On W ON E.EmpID = W.EmpID
-> RIGHT JOIN Project P ON W.ProjID = P.ProjID;
+-----+-----+
| EName  | ProjName          |
+-----+-----+
| Arun   | ERP Upgrade       |
| Bala   | ERP Upgrade       |
| Chitra | Recruitment Drive |
| Divya  | NULL              |
| Esakki | Audit 2025        |
| Farook | NULL              |
| Gowri  | Q3 Sales Push     |
| Harish | NULL              |
| Indra   | ERP Upgrade       |
| Jaya   | NULL              |
+-----+-----+
10 rows in set (0.011 sec)
```

```
mysql> -- CROSS JOIN
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT E.ENAME, P.ProjName
-> FROM Employee E
-> CROSS JOIN Project P;
```

ENAME	ProjName
Arun	Q3 Sales Push
Arun	Audit 2025
Arun	Recruitment Drive
Arun	ERP Upgrade
Bala	Q3 Sales Push
Bala	Audit 2025
Bala	Recruitment Drive
Bala	ERP Upgrade
Chitra	Q3 Sales Push
Chitra	Audit 2025
Chitra	Recruitment Drive
Chitra	ERP Upgrade
Divya	Q3 Sales Push
Divya	Audit 2025
Divya	Recruitment Drive
Divya	ERP Upgrade
Esakki	Q3 Sales Push
Esakki	Audit 2025
Esakki	Recruitment Drive
Esakki	ERP Upgrade
Farook	Q3 Sales Push
Farook	Audit 2025
Farook	Recruitment Drive
Farook	ERP Upgrade
Gowri	Q3 Sales Push
Gowri	Audit 2025
Gowri	Recruitment Drive
Gowri	ERP Upgrade
Harish	Q3 Sales Push
Harish	Audit 2025
Harish	Recruitment Drive
Harish	ERP Upgrade
Indra	Q3 Sales Push
Indra	Audit 2025
Indra	Recruitment Drive
Indra	ERP Upgrade
Jaya	Q3 Sales Push
Jaya	Audit 2025
Jaya	Recruitment Drive
Jaya	ERP Upgrade

```
40 rows in set (0.007 sec)
```


Q6. Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

```
mysql> CREATE OR REPLACE VIEW DeptSalaryView AS
-> SELECT D.DeptName,
->         COUNT(E.EmpID)    AS TotalEmployees,
->         AVG(E.Salary)     AS AvgSalary
-> FROM Department D
-> LEFT JOIN Employee E ON D.DeptID = E.DeptID
-> GROUP BY D.DeptName;
Query OK, 0 rows affected (0.014 sec)
```

```
mysql> -- Query the view
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT * FROM DeptSalaryView
-> ORDER BY AvgSalary DESC;
```

```
+-----+-----+-----+
| DeptName | TotalEmployees | AvgSalary |
+-----+-----+-----+
| Finance  | 2 | 54500.000000 |
| IT       | 3 | 53666.666667 |
| HR       | 3 | 53500.000000 |
| Sales    | 2 | 46500.000000 |
+-----+-----+-----+
4 rows in set (0.015 sec)
```

Q7. Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

Step 1 — Get the set of all course IDs offered:

$AllCourses = \pi_{CourseID}(Course)$

Step 2 — Get (student, course) pairs from Enrollment:

$StudentCourses = \pi_{SID, CourseID}(Enrollment)$

Step 3 — Divide to find students enrolled in every course:

$FullyEnrolled = StudentCourses \div AllCourses$

Step 4 — Join back to Student to get names:

$Result = \pi_{Name}(FullyEnrolled \bowtie Student)$

Combined Expression:

$\pi_{Name} ((\pi_{SID, CourseID}(Enrollment) \div \pi_{CourseID}(Course)) \bowtie Student)$

Q8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

For Insert:

```
mysql> -- INSERT: add a new student (valid DeptID required by FK constraint)
Query OK, 0 rows affected (0.000 sec)

mysql> INSERT INTO Student (SID, Name, DOB, DeptID) VALUES (6, 'Vikram', '2003-03-12', 1);
Query OK, 1 row affected (0.011 sec)
```

Integrity Constraint for Insert:

```
mysql> -- Attempt an INSERT that violates the FK constraint (should fail/error)
Query OK, 0 rows affected (0.000 sec)

mysql> INSERT INTO Student (SID, Name, DOB, DeptID) VALUES (7, 'Wrong', '2003-01-01', 99);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`dbms_lab`.`Student`, CONSTRAINT `Student_ibfk_1` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`))
```

For Update:

```
mysql> -- UPDATE: change a student's department
Query OK, 0 rows affected (0.000 sec)

mysql> UPDATE Student SET DeptID = 2 WHERE SID = 6;
Query OK, 1 row affected (0.013 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

For Delete:

```
mysql> DELETE FROM Student WHERE SID = 6;
Query OK, 1 row affected (0.002 sec)
```

Integrity Constraint for Delete:

```
mysql> DELETE FROM Department WHERE DeptID = 1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint` (`DeptID`))
```

Final Output:

```
mysql> SELECT * FROM Student;
+-----+-----+-----+-----+
| SID | Name   | DOB       | DeptID |
+-----+-----+-----+-----+
| 1   | Ravi   | 2003-05-10 | 1      |
| 2   | Meena  | 2003-08-21 | 1      |
| 3   | Suresh | 2002-11-02 | 2      |
| 4   | Priya  | 2003-01-15 | 3      |
| 5   | Anand  | 2002-07-30 | 4      |
+-----+-----+-----+-----+
5 rows in set (0.000 sec)
```

Q9. Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

```
mysql> -- SQL equivalent
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT E.*
-> FROM Employee E
-> WHERE E.Salary > 50000
-> AND E.DeptID = (SELECT D.DeptID FROM Department D WHERE D.DeptName = 'IT');
+-----+-----+-----+-----+
| EmpID | EName | Salary | DeptID |
+-----+-----+-----+-----+
| 101   | Arun  | 55000.00 | 1 |
| 109   | Indra  | 58000.00 | 1 |
+-----+-----+-----+-----+
2 rows in set (0.008 sec)
```

Q10. Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

```
mysql> -- Query 1 (JOIN): list each passenger with their booked flight route
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT P.PName, F.Origin, F.Destination, F.FlightDate
-> FROM Booking B
-> JOIN Passenger P ON B.PassID = P.PassID
-> JOIN Flight F ON B.FlightID = F.FlightID;
+-----+-----+-----+-----+
| PName | Origin | Destination | FlightDate |
+-----+-----+-----+-----+
| Kiran | Chennai | Delhi | 2025-02-01 |
| Lavanya | Chennai | Delhi | 2025-02-01 |
| Manoj | Chennai | Mumbai | 2025-02-02 |
+-----+-----+-----+-----+
3 rows in set (0.010 sec)
```

```
mysql> -- Query 2 (JOIN): count how many passengers are booked on each flight
Query OK, 0 rows affected (0.000 sec)
```

```
mysql> SELECT F.FlightID, F.Origin, F.Destination, COUNT(B.BookingID) AS NumPassengers
-> FROM Flight F
-> LEFT JOIN Booking B ON F.FlightID = B.FlightID
-> GROUP BY F.FlightID, F.Origin, F.Destination;
+-----+-----+-----+-----+
| FlightID | Origin | Destination | NumPassengers |
+-----+-----+-----+-----+
| 1 | Chennai | Delhi | 2 |
| 2 | Chennai | Mumbai | 1 |
| 3 | Delhi | Bangalore | 0 |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

```
mysql> -- Query 3 (SUB-QUERY): passengers who have NOT booked any flight
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT PName
-> FROM Passenger
-> WHERE PassID NOT IN (SELECT PassID FROM Booking);
Empty set (0.009 sec)

mysql> -- Query 4 (SUB-QUERY): flights with more than 1 booking
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT FlightID
-> FROM Flight
-> WHERE FlightID IN (
->     SELECT FlightID FROM Booking
->     GROUP BY FlightID
->     HAVING COUNT(*) > 1
-> );
+-----+
| FlightID |
+-----+
|          1 |
+-----+
1 row in set (0.002 sec)

mysql> -- Query 5 (VIEW): create and query a view of full booking details
Query OK, 0 rows affected (0.000 sec)

mysql> CREATE OR REPLACE VIEW BookingDetails AS
-> SELECT B.BookingID, P.PName, F.Origin, F.Destination, F.FlightDate, B.SeatNo
-> FROM Booking B
-> JOIN Passenger P ON B.PassID = P.PassID
-> JOIN Flight F ON B.FlightID = F.FlightID;
Query OK, 0 rows affected (0.011 sec)

mysql> SELECT * FROM BookingDetails;
+-----+-----+-----+-----+-----+-----+
| BookingID | PName   | Origin | Destination | FlightDate | SeatNo |
+-----+-----+-----+-----+-----+-----+
|          1 | Kiran   | Chennai | Delhi       | 2025-02-01 | 12A    |
|          2 | Lavanya | Chennai | Delhi       | 2025-02-01 | 12B    |
|          3 | Manoj   | Chennai | Mumbai     | 2025-02-02 | 5C     |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.007 sec)
```